Challenge 03: Autonomous Legacy Modernization Agent

Team Neural Nexus | Approach, solution and innovations | Draft for team review

1. The problem

Old applications still run important business. The hard part of modernizing them is not rewriting the code. It is understanding what the system really does: documentation is incomplete, developers have moved on, and critical business rules exist only inside the code. The AI must read the legacy evidence and produce an evidence-based modernization strategy. It must never recommend deleting or rewriting code only because it uses old technology.

Required demonstrations: application understanding, business-rule extraction, dependency discovery, database dependency analysis, obsolete-code identification, undocumented-behaviour detection, technical-debt analysis, modernization risk analysis, modernization alternatives, and migration planning.

Critical requirement: tell apart code that looks obsolete from code that implements an undocumented but critical business rule.

2. Our solution in simple terms

We build a tool that reads the evidence like a detective. It finds what each part of the system does, which parts depend on it, which database tables it reads or writes, and which if-rules control behaviour. Then it checks each rule against the docs, incidents and tickets. A rule that shows up in incidents or tickets, or that writes data, is treated as critical even when nobody documented it.

Every component gets one of three verdicts. Each verdict comes with the reasons behind it:

Verdict	When it applies	What happens
modernize	Old technology, nothing depends on it, no critical rule, no incidents	Safe to replace in a later phase
review	Old but critical, or high risk (3 or more points of risk)	Kept as it is; a human signs off first
keep	Modern and low risk	No action

3. How the analysis works

1. **Ingest:** accept code, schema, config, interfaces, docs, incidents and tickets.
2. **Map dependencies:** imports and calls between code files, plus table reads and writes from SQL and CREATE TABLE statements.
3. **Extract business rules:** every if, elif or when line that compares values, with comments removed.
4. **Check rules:** documented if the rule's terms appear in the docs; critical if linked to an incident or ticket, if its component writes to a table, or if two or more components depend on it.
5. **Score and verdict:** heat map with a risk score, obsolete markers and reasons.
6. **Report:** current state, tech debt, risks, alternatives, target architecture, phased roadmap and test strategy.

Phase 1 is rule-based (pattern matching and scoring). It is fast, works offline and is easy to explain. AI-written text is added later on top of these facts, so every claim can point to a source line.

4. Worked example: our sample legacy app

File	What it shows	Expected verdict
legacy_billing.py	Old HTTP library import; a late-fee rule (days_late greater than 45) with no documentation; writes to invoices	review
payments.py	Large-order approval rule (order total of 10000 or more), backed by an incident	review
old_report.py	Marked deprecated; nothing calls it	modernize

The undocumented late-fee rule is flagged as critical and undocumented. That is exactly the case the brief asks us to catch, and it is why old_report.py is the only safe removal candidate.

5. API contract (v1)

Method and path	Purpose	Returns
POST /api/analyses	Start an analysis from submitted artifacts	id and status (done)
GET /api/analyses/{id}	Check an analysis	id, name and status
GET .../business-rules	Business-rule catalogue	Rules with source line, evidence, confidence, critical, documented
GET .../dependencies	Dependency map	Nodes (code and tables) and edges (imports, reads, writes)
GET .../heatmap	Modernization heat map	Component, risk, tech debt, obsolete flag, verdict, reasons
GET .../report	Strategy report	Current state, tech debt, risks, alternatives, architecture, roadmap, tests

6. Build plan and roles

Core first, then extras. Each step is its own branch and pull request, reviewed before merge.

Step	What	Owner
1	Analysis engine (dependencies, rules, verdicts, report)	You
2	API endpoints from the contract	You
3	Sample legacy app and automated tests	Nagukan, with You
4	Frontend: heat map, rule list, dependency view	Nishanth
5	Pitch deck, demo script and presentation	Preneeth and Ajmal
Extra	AI-written explanations (Claude API), zip upload, PDF report, innovations in section 8	Shared, after core

7. Existing products and what we add

We checked public material on the main tools. They extract rules, map dependencies and translate code. None of the sources we read explain why a component is safe to remove, or show the evidence behind a keep, review or modernize call. That gap is what we target.

Product	What it does (per public sources)	Gap for this challenge
AWS Transform	Extracts COBOL and JCL business rules with source-line references; builds a deterministic baseline before applying AI	Focused on mainframe code; keep/review/remove verdict with incident evidence not described
CAST Imaging (MCP server)	Semantic analysis of application structure; gives AI agents architectural context; used on a 40-year-old mainframe application	Strong structure view; no evidence-based verdict on what to remove
IBM watsonx Code Assistant for Z	Business-rule discovery for COBOL with AI-drafted rule purpose and logic; documentation generation	COBOL-focused; does not weigh incidents or tickets

Product	What it does (per public sources)	Gap for this challenge
Imogen (Mechanical Orchard)	Works with AWS Transform domains and rules	Depends on the AWS Transform rule set
Legacy modernizer skills (Claude Code)	Characterization tests, golden-master pattern, strangler fig with staged traffic	Process guidance rather than an analysis engine

Moderne, vFunction, and Google or Microsoft tools were not verified in our research, so they are not compared here.

8. Our innovations

Ranked by value for effort:

#	Innovation	What it does	Why it matters
1	Evidence chain per verdict	Each verdict links to the source line, incident and ticket behind it	Directly meets the critical requirement
2	Auto-generated characterization tests	For each critical rule, writes a pytest golden-master stub using boundary values from its numbers (for example 45 and 46 for days_late)	Covers regression coverage
3	Incident archaeology	Links incidents and tickets to the rule or component they explain	Finds the why behind undocumented rules
4	AI with citations	Facts come first; Claude writes plain-English rule text; any claim without a source citation is dropped	Reduces hallucination risk
5	Blast-radius view	Select a component to see its dependents, tables, rules and incidents affected by a change	Makes dependency accuracy visible
6	Human sign-off queue	The business owner approves or rejects each critical rule; phase 3 unlocks only after sign-off	Respects the no-deletion guardrail
7	Dependency-ordered roadmap	Phases ordered leaf-first so strangler seams are safe to cut	Improves modernization feasibility

9. How we meet the success measures

Success measure	Our feature
Business-rule extraction	Rules pulled from conditions, each with its source line
Dependency accuracy	Import, call and table edges; checked against the sample app
Modernization feasibility	Verdicts with reasons; leaf-first roadmap
Risk identification	Risk score and a review list
Architecture quality	Target architecture built around the strangler approach
Regression coverage	Characterization test stubs for every critical rule

10. Guardrails and known limits

- Old technology alone never gets the modernize verdict. It also needs no critical rule, no incidents and no dependents.
- Phase 1 rules are heuristic. They can miss rules written in unusual syntax and can over-flag. Human review is part of the design.
- Call and table detection uses patterns, so it will not parse every language.
- The data store is in memory, so data is lost on restart. That is fine for the demo.
- Vendor claims in section 7 come from public marketing and have not been independently verified.

11. Questions for teammates

1. Is Python with FastAPI still the right backend for everyone?

2. Should the demo use only the sample app, or also real code if we can get some?
3. Which two innovations from section 8 should we build first after the core?
4. Who presents each part of the demo: problem, approach, live demo, innovations?
5. What does the frontend need from the API contract that is not in it yet?

12. Sources

- AWS Transform, mainframe traceability: https://aws.amazon.com/blogs/migration-and-modernization/reimagine-mainframe-applications-with-traceability-and-speed-using-aws-transform/
- CAST Imaging MCP server: https://www.castsoftware.com/mcp
- CAST case study, IBM Consulting and a 40-year-old mainframe application: https://learn.castsoftware.com/case-studies/ibm-consulting-grounds-ai-to-accelerate-understanding-of-a-40-year-old-mainframe-application
- IBM watsonx Code Assistant for Z, business-rule discovery: https://community.ibm.com/community/user/blogs/stephanie-marino/2025/11/11/accelerate-understanding-and-documentation-on-ibm
- Imogen and AWS Transform: https://www.mechanical-orchard.com/insights/imogen-now-integrated-with-aws-transform
- Strangler fig and characterization tests: https://codex.danielvaughan.com/2026/06/14/codex-cli-legacy-code-modernisation-strangler-fig-execplan-parity-testing-migration-workflows/
- Legacy modernizer skill (golden master, staged traffic): https://claudeskills.info/skills/jeffallan/claude-skills/legacy-modernizer/